A08 Midterm: Regex Model vs Gemini Model, Run Like a Real Experiment

OPIM 5512 · Module 3 · Solo · Due Sunday, Nov 1, 11:59 PM (extended) · Builds on Lab 4 (your regex model) and A07 (your Gemini fields)

> **The one-sentence version: can large-language-model features beat regular expressions at pricing used cars, and are they worth the money?** You answer it the way a data science team does at work: two tuned models on the same cars, both deployed, a fair live test on new cars, the cost of each, and a 2-page report your manager can act on.

See it done first. Dr. Dave ran an offline version of this experiment on his own cars: his notebook, every chart included and his one-page memo. On his cars the Gemini model was about $150 better per car, at about $2 per 1,000 listings. Your report goes further: a deployed Gemini model, a live test on new cars, and your own cost proof. Your cars, your fields, your answer may differ. That is the point.

What you do

1. **Build one data set, two feature sets.** The regex model uses the fields your Lab 4 regular expressions pull out of each ad (`listings_master.csv`). The Gemini model uses the fields Gemini on Vertex AI pulls out of the same ads (A07, `listings_master_llm.csv`). Join them on `post_id` so both feature tables have the same cars: if you have 400 cars, both have the same 400 rows.
2. **Tune both models.** Same recipe for both (same model families, grids, seed and cross-validation); only the feature list changes. Get the best fit you can for each.
3. **Deploy the Gemini model on Google Cloud.** Your regex model is already live from Lab 4. **Deployment is the most important skill in this midterm**: a model that only lives in Colab helps nobody. There is no step-by-step guide for this part on purpose. Use **Claude as your teammate**: show it your repo, your model card and your errors, and work it out together. Put the key prompts you used, and what you changed, in your appendix.
4. **Run a fair live test.** Once the Gemini model is live, let it score the new cars that come in, for anywhere from **1 to 7 days**. Then compare both models on the **same** new cars, which neither model saw in training, scored by both **frozen** models.
5. **Explain both models.** At least one SHAP plot for each model. What drives price in each? Did any Gemini-only field matter?
6. **Work out the cost.** What does each model cost to run once it is deployed? Show your calculation and the proof.
7. **Recommend one.** Should the company run the regex model or the Gemini model, and why?

> ### Two hints
>
> 1. **Ship the Gemini model by Wednesday, October 28**, so you have at least one full day of new cars before the deadline.
> 2. `predict-shap` runs one model at a time, so shipping the Gemini model replaces your regex model in production. That is fine: your bucket keeps a copy of every model you ever shipped in `models/history/` . Score the same new cars with your frozen regex model from there, and you have a fair head-to-head.

The starter notebook

Open the **midterm starter notebook** (**File → Save a copy in Drive**), type your project ID and bucket name in Part 0, and run it. It reads both tables from your bucket, finds your new Gemini columns by itself, joins and cleans both tables the same way, and tunes all three model families for each feature set. It also has SHAP, permutation importance and partial dependence plots.

Ideas, not requirements: the starter's **25 paired folds** ("is the difference real, or luck?") and the **ablation** (how much came from Gemini reading the same 4 fields better vs from your new fields) make strong evidence for your report. Use them if they help your argument.

The deliverable: a 2-page report for your manager

- **2 pages of text** (12 pt, single spaced), written for a manager. Figures, tables and screenshots go on extra pages after the 2 pages and do not count toward the limit. Number them (Figure 1, Figure 2...) and refer to them in the text.
- **Screenshots are your proof.** If a claim has no screenshot or notebook output behind it, it does not get credit.
- See Dr. Dave's memo for the tone.

What the report covers, and the proof we need

#	Section	What to show	Proof (screenshot or notebook output)
1	The data set	How many cars; the same cars in both tables; the regex features and the Gemini features you used	Shape of the joined table; the two feature lists
2	Tuning	The models and grids you tried for each feature set, cross-validation error, test error, the champion for each	The tuning scoreboard from the notebook
3	Deployment	The Gemini model live on Google Cloud, next to your Lab 4 regex model; how you got it there	Both model cards; `preds_master.csv` with your Gemini model's `model_version` scoring new cars; a successful `predict-shap` run; your key Claude prompts
4	The live test	The dates of your window (1 to 7 days), how many new cars, and each model's average error on the **same** cars. Which model won, and by how much?	Head-to-head table and one plot (actual vs predicted, or error by day)
5	Interpretability	At least one SHAP plot for each model. What drives price in each? Did any Gemini-only field matter?	SHAP plots (bar or beeswarm)
6	Cost	What each model costs to run per 1,000 cars and per month at your scraping rate, with the calculation written out	Billing → Reports filtered to your project, grouped by service (Vertex AI vs Cloud Run functions), for the dates you used, next to how many listings Gemini read in those dates
7	Recommendation	Regex or Gemini, and why: dollars of accuracy per car against dollars of cost. What would change your mind?	Refers back to sections 4 to 6

How to do the cost math (section 6)

Both models pay for scraping, regex extraction and `predict-shap`. Only the Gemini model also pays for `extractor-llm` and `materialize-llm`, plus the Vertex AI tokens. So:

1. **Extra cost of the Gemini model in your window** ≈ what Billing shows for Vertex AI (plus the extra function runs) for those dates.
2. **Cost per listing** = that amount ÷ the number of listings Gemini read in the same dates.
3. **Scale it:** cost per listing × 1,000, and cost per listing × new listings per hour × 24 × 30 = cost per month.
4. **Check that it is believable:** tokens per ad (characters ÷ 4) × Google's price per token for gemini-2.5-flash.

Billing can lag a day, so take the screenshot at least a day after your window ends.

Rubric (100 points)

Criterion	Pts	Full credit	Partial credit	Little or no credit
1. Same-car data set	10	Both feature sets on the same cars (counts match, joined on `post_id`), features listed, junk rows handled the same way for both	Joined, but counts or cleaning differ between the two models and the report doesn't say so	No joined data set, or the models used different cars
2. Tuning both models	15	Same recipe for both (same model families, grids, seed and CV); CV and test error reported; a clear champion for each	Tuned, but unevenly (for example, a bigger grid for one), or the errors are missing	Default settings, or only one model tuned
3. Gemini model deployed	15	Gemini model live and scoring new cars; screenshots of both model cards, `preds_master` with the Gemini version and a successful `predict-shap` run; key Claude prompts in the appendix	Shipped, but no proof that it scored new cars	Not deployed
4. Fair live comparison	20	Window dates and number of new cars stated; both frozen models scored the same unseen cars; average error for each, the gap, and one plot; the winner stated plainly	A comparison, but on different cars or windows, or there is no plot	No live comparison
5. Interpretability	10	At least one SHAP plot per model, read correctly; the top drivers compared; says whether the Gemini fields earned their place	SHAP plots with little or no explanation	No SHAP plots
6. Cost, with proof	15	Billing screenshot with dates; the calculation written out (cost per listing, per 1,000, per month); a token-based check; both models covered	A cost number with no screenshot or no worked calculation	Cost not addressed, or only copied from Dr. Dave's memo
7. Business recommendation	10	A clear call (regex or Gemini) that weighs the accuracy gain in dollars against the cost, and says what would change the call	A call, but it doesn't follow from the numbers	No recommendation
8. Report quality	5	2 pages of text, numbered figures that the text refers to, numbers that match the notebook, written for a manager	Over length, or figures not referenced	Hard to follow, or numbers contradict the notebook

Submit (HuskyCT → Submit A08 Midterm)

- Your **report** (PDF): 2 pages of text + numbered figure pages
- Your **notebook** (**File** → **Download** → **.ipynb**), run top to bottom

If you get stuck

You see	Do this
"No new Gemini columns found"	Your A07 Edit 3 (`out_record`) is missing: A07 Step 2, then Step 4 to re-read old cars
Very few new cars	Check your scraper is still running; tell Dr. Dave
`predict-shap` says "your model needs columns ..."	Your model card needs a `data_key` that points at `structured/datasets/listings_master_llm.csv`
`predict-shap` ignores your new fields	Sync your fork (Sync fork → Update branch, never Discard commits) so `predict-shap` reads the feature list from your model card
Anything else	Paste the cell, the full error and what you expected into Claude